System for Automatic Generation of Educational Quizzes Based on Ontology and SOLO Taxonomy

Uroš Petrašković [1]
[1] Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad, Serbia

author's e-mail: petraskovic.uros@gmail.com

ORCID IDs

1. Uroš Petrašković - https://orcid.org/0009-0005-5193-5089

Abstract

This paper describes the development of a system that can automatically create educational quizzes. The system uses artificial intelligence running locally on the user's computer, combined with semantic web technology and the SOLO taxonomy to generate questions that match different levels of student understanding. Some of the main features include a chatbot that helps students learn, a tool for running SPARQL queries against the knowledge base, and the ability to export the course ontology as an OWL file that can be opened in Protégé. The paper also includes a hands-on evaluation where the questions were manually reviewed to check how good they are and whether the wrong answer choices are tricky enough to actually test student knowledge.

Keywords

Question Generation, Education, SOLO Taxonomy, Ontology, Automatic Generation, Artificial Intelligence, AI, SPARQL, Question Evaluation, Protégé, Distractors, Distractor Quality

1. Introduction

Education has always been important, and with technology becoming a bigger part of our lives, there is a growing need for tools that can help both teachers and students. One of the challenges teachers face is creating quizzes and tests. It takes a lot of time to come up with good questions, especially when you want to test students at different levels of understanding. A simple recall question is very different from a question that asks students to apply what they learned to a new situation.

This project tries to help with that problem. The idea is to build a system that can take educational content, like a PDF of lecture notes, and automatically generate quiz questions from it. But not just random questions - questions that are organized according to the SOLO taxonomy [1], which is a way of categorizing how deeply someone understands a topic. The system also includes a chatbot that students can talk to if they have questions about the material, and it stores all the knowledge in a structured way using ontologies [2] so that connections between concepts can be explored.

The rest of this paper is organized as follows. Section 2 explains motivation. Section 3 looks at what other researchers and companies have done in this area. Section 4 describes how the system is built from a technical perspective. Section 5 goes into detail about the SOLO taxonomy and the ontology that stores the knowledge. Section 6 covers the implementation, meaning the actual code and how everything works together. Section 7 presents an evaluation where I checked the quality of the generated questions by hand. Finally, Section 8 wraps everything up with conclusions and ideas for future work.

2. Motivation

The first and most obvious one is that creating quizzes manually is time-consuming. Teachers already have a lot on their plate, and spending hours writing questions for every topic is not always realistic. According to some research, writing a single good multiple-choice question can take 15 to 30 minutes if you want to do it properly.

Multiply that by the number of questions needed for an entire course, and it becomes clear why automation could help.

Another motivation is personalization. Students learn at different paces and have different strengths. A quiz that is too easy for one student might be too hard for another. By using the SOLO taxonomy, the system can generate questions at different cognitive levels, from simple recall to complex application. This makes it possible to create quizzes that are tailored to what each student needs to practice.

Project is not dependant on expensive cloud services. Many AI-powered tools today require API keys and charge money for every request. For a student project, or for schools with limited budgets, that is not always an option. By using Ollama [3] to run a language model locally, the system can work without internet access and without ongoing costs. This also means that student data stays on the local machine, which is better for privacy.

3. Review of Existing Solutions

The field of Automatic Question Generation (AQG) has evolved significantly over the past two decades. One of the foundational works is by Mitkov and Ha [4] (2003), who developed a system that used natural language processing rules to transform declarative sentences into questions. Their approach could take a sentence like "Photosynthesis occurs in chloroplasts" and generate "Where does photosynthesis occur?" While effective for simple factual content, these rule-based systems struggled with complex material and often produced grammatically awkward questions.

The challenge of generating good distractors has received special attention. Liang et al. [5] (2018) proposed using a "learning to rank" approach where the system generates many candidate distractors and then ranks them by how plausible they are. Their key insight was that good distractors should be semantically related to the correct answer but clearly wrong upon careful reading. For example, if the correct answer is "mitochondria," good distractors might be "ribosomes" or "nucleus" rather than "banana."

More recently, Kasneci et al. [6] (2023) published a comprehensive review of using large language models like ChatGPT in education. They highlighted both the potential and the risks, noting that while these models can generate diverse questions quickly, they also raise concerns about accuracy, bias, and the need for teacher oversight. Importantly, they pointed out that cloud-based solutions create privacy issues when handling student data, which is one reason why local AI solutions like the one in this project are valuable.

Lister et al. [7] (2006) examined how the SOLO (Structure of Observed Learning Outcomes) taxonomy could be applied to classify and evaluate programming questions. Their seminal work, "Not seeing the forest for the trees: novice programmers and the SOLO taxonomy," revealed a critical insight: many programming education assessments focus too heavily on low-level skills (identifying syntax errors, recognizing code fragments) while neglecting higher-order thinking skills (integrating concepts, designing solutions). They demonstrated that exam questions could be systematically classified into SOLO levels, from uni-structural questions that test single concepts like "What does this variable store?" to extended abstract questions like "Design a program that solves this complex problem using multiple data structures." Their framework provides a structured way to ensure that questions across all complexity levels are included.

4. System Architecture

The system is split into three main parts: the backend, the frontend, and the AI layer. Each part has a specific job, and they communicate with each other through a REST API [8]. This kind of separation makes it easier to work on one part without breaking the others.

4.1. Backend

The backend is written in Python using Flask [8], handles all the business logic, like storing lessons in the database, generating questions, and managing quizzes. The database is SQLite [10], which stores everything in a single file. This makes the system easy to set up because you do not need to install a separate database server.

The backend is organized into several layers. The repository layer handles database operations using SQLAlchemy, which is an ORM that lets you work with the database using Python objects instead of writing raw

SQL. The service layer contains the actual logic for things like parsing content, generating questions, and managing the ontology. There are separate services for lessons, questions, quizzes, the chatbot, and SPARQL [11] queries.

4.2. Frontend

The frontend is a React [12] application that provides the user interface. It has different components for different tasks. The CourseManager lets you create and organize courses. The LessonManager handles uploading and parsing PDF files. The QuestionGenerator triggers the AI to create questions. The QuestionBank shows all the generated questions and lets you edit or delete them. The QuizBuilder lets you put questions together into a quiz, and the QuizSolver is where students actually take the quiz.

There is also a ChatBot component that opens a chat window where students can ask questions about the course material. And the SPARQLQueryTool is a more advanced feature that lets users write their own queries to explore the ontology.

4.3. AI Layer

The AI layer uses Ollama. The system is configured to use the Qwen 2.5 model with 14 billion parameters. This model is good at understanding and generating text, and it runs on the local machine without needing an internet connection or API keys. The backend communicates with Ollama through HTTP requests, sending prompts and receiving generated text.

5. SOLO Subsystem and Ontology

There are four levels: Unistructural, Multistructural, Relational, and Extended Abstract. Each level corresponds to a different type of question. Unistructural questions ask about a single fact. Multistructural questions ask about multiple facts. Relational questions ask how things connect. And Extended Abstract questions ask students to apply what they learned in a new context. You can see Figure 1 below below for visualization.

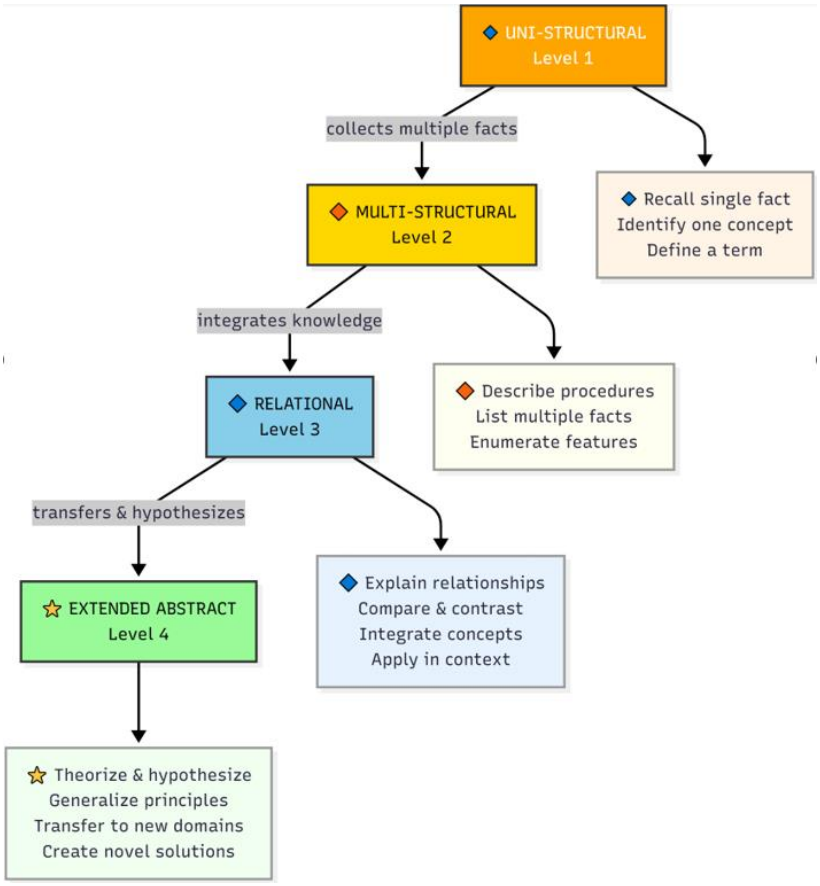

Figure 1 - SOLO question structure diagram

5.1. How SOLO is Used in Question Generation

When the system generates questions, it uses different prompts for each SOLO level. The prompts are carefully designed to guide the AI toward creating appropriate questions. For example, the prompt for Unistructural questions explicitly says to focus on a single fact and avoid asking about connections.

5.2. The Ontology

The system uses an ontology to represent the structure of knowledge. The ontology is written in Turtle format, It defines classes like Course, Lesson, Section, LearningObject, and Question, and properties that link them together.

When a lesson is parsed, the system creates entries in the ontology for all the concepts it finds. It also tries to identify relationships between concepts, like prerequisites or related topics. This information can be used to generate better questions and to help students understand how different parts of the material connect.

5.3. SPARQL Queries and OWL Export

The system includes a SPARQL query interface. Users can write queries to find specific information, like all questions at a certain SOLO level, or all learning objects that are related to a particular concept. The interface includes some example queries to help users get started.

There is also an option to export the ontology as an OWL file. It is a standard format that can be opened in tools like Protégé. Protégé can visualize the knowledge graph, showing nodes and edges that represent concepts and their relationships.

6. System Implementation

Here is how a typical workflow goes: First, the user uploads a PDF file containing lecture notes or a textbook chapter. The backend extracts the text from the PDF using a library called PyPDF2. Then the text is sent to the AI model, which parses it into sections and learning objects. These are stored in the database with the hierarchical structure: Course contains Lessons, Lessons contain Sections, and Sections contain Learning Objects.

After generating all the learning objects then the user can build Ontology relationships based on those objects which can be exported as an owl file. Those relationships are used to improve question generation.

Next, the user can trigger question generation. The system goes through the learning objects and ontologies and generates questions at different SOLO levels. Each question is saved to the database along with its options, correct answer, and explanation. User can review the questions, edit them if needed, and then build quizzes by selecting which questions to include.

User can access the quizzes through the QuizSolver component. They answer the questions, and when they submit, they see their score and can review which questions they got right or wrong. The explanations help them understand why the specific answer is correct. You can see the structure in detail on figure 2.

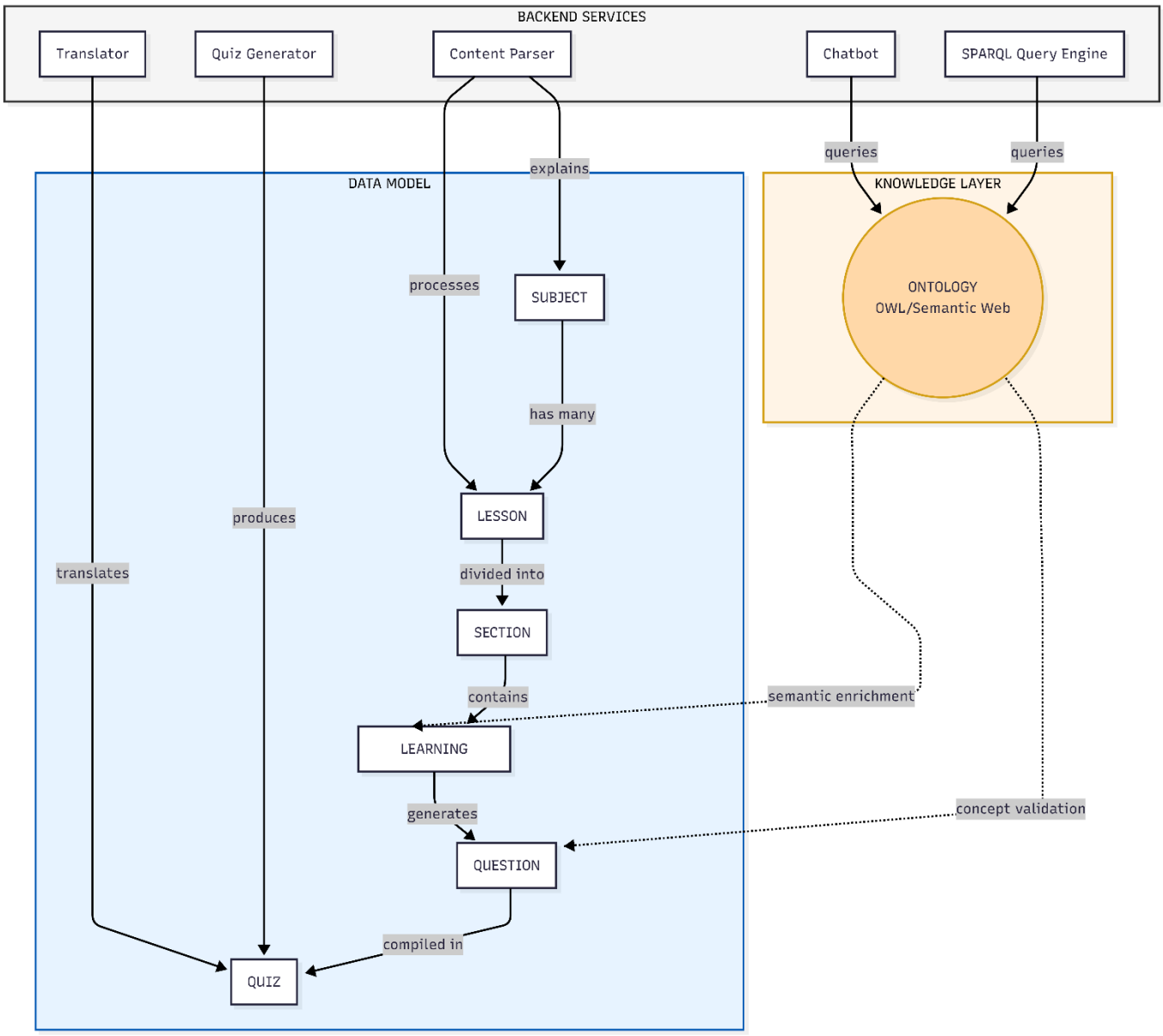

Figure 2 – Workflow diagram

6.1. The Chatbot

The chatbot uses a Retrieval-Augmented Generation (RAG) approach. When a student asks a question, the system first searches the database for relevant content. It looks through lessons, sections, and learning objects to find text that matches the question. This context is then included in the prompt sent to the language model, so the response is grounded in the actual course material.

6.2. Multi-Language Support

The system includes a translation manager that can translate quizzes into different languages. This uses the same AI model but with prompts for translation. The interface remembers the user's language preference, so if a student switches to Serbian, for example, the questions will be shown in Serbian if translations are available.

7. Evaluation and Quality

To evaluate the system, I did a manual review of the generated questions. The goal was to check two things: first, whether the questions match their assigned SOLO level, and second, whether the distractors are good enough that students cannot just guess the right answer by eliminating obviously wrong options.

7.1. SOLO Level Accuracy

The questions generally matched their assigned SOLO levels. Unistructural questions asked about single facts. Multistructural questions asked about multiple aspects without requiring integration. Relational questions

required understanding connections. And Extended Abstract questions presented new scenarios that required applying and synthesizing knowledge. The prompts seem to be doing their job of guiding the AI toward the right type of question. The explicit definitions of each SOLO level in the prompts help the model understand what kind of question to generate.

7.2. Unistructural question example

Unistructural Example: "The Program Counter (PC) holds the memory address of which type of entity?" The options were: A) The next instruction to be executed, B) The current executing instruction, C) All instructions in memory, D) Data variables. The correct answer is A. This item successfully isolates a single, fundamental concept—the specific function of the Program Counter. The distractors show variable quality. Option B is particularly effective, exploiting temporal confusion between "next" and "current" that targets students with incomplete understanding of the fetch-execute cycle. Option C addresses a novice misconception about PC scope, though it may be somewhat obvious to students who understand that registers hold single values. Option D, while testing the distinction between instruction and data addresses, is the weakest distractor—students with even basic architecture knowledge would likely recognize that the PC relates to instructions rather than data variables, making this option easily eliminated.

7.3. Multistructural question example

Multistructural Example: "Which of the following statements correctly describe both User-Level Threads (ULT) and Kernel-Level Threads (KLT)?" The options were: A) ULT are managed by application, KLT by OS; ULT can provide finer control but may be less efficient than KLT, B) Both ULT and KLT require explicit synchronization mechanisms to prevent race conditions, C) ULT provides more system resources allocation units compared to KLT, D) Kernel manages both ULT and KLT, providing equal efficiency and control. The correct answer is A. Option B is well-constructed, appearing plausible since synchronization is genuinely relevant to threading, though it incorrectly universalizes a concern that varies by implementation. Option D effectively targets the misconception that kernel management equalizes all thread characteristics. Option C, however, is the weakest distractor—the phrasing "provides more system resources allocation units" is awkwardly technical and vague enough that students might eliminate it based on unclear wording rather than actual knowledge of thread resource allocation.

7.4. Relational question example

Relational Example: "How does test automation contribute to the effectiveness of regression testing?" The options were: A) Test automation only improves the speed of functional testing without impacting regression testing, B) Test automation replaces manual testing entirely, making regression testing unnecessary, C) Test automation allows for faster execution and identification of changes in functionality, enhancing the thoroughness of regression tests, D) Test automation focuses on creating new test cases rather than running existing ones efficiently. The correct answer is C. The question successfully requires students to connect two concepts—understanding both what regression testing entails (re-executing existing tests to detect unintended changes) and how automation specifically addresses its challenges. The distractors show reasonable quality. Option B captures an extreme misconception about automation replacing human judgment entirely, while Option A creates an artificial boundary between functional and regression testing that might appeal to students with lesser understanding. However, Option D is problematic. The claim that "test automation focuses on creating new test cases" doesn't align with any common misconception about automation—most students would recognize that automation executes tests rather than generates them. This makes the distractor too easily eliminated. Additionally, Option A's specificity about "only functional testing" may be overly obvious, as students familiar with basic automation concepts would recognize that automated tests can serve multiple purposes. Stronger distractors might address misconceptions about automation coverage limitations or the false belief that automated tests detect different types of defects than manual regression testing.

7.5. Extended abstract question example

Extended Abstract Example: "In a distributed computing environment, if a main process spawns multiple child processes across different nodes, what mechanism ensures that all child processes receive the correct system resources and do not interfere with each other?" The options were: A) Implementing inter-process communication (IPC) protocols to manage resource allocation and synchronization, B) Increasing the priority of the parent process over its children to control their execution flow, C) Allocating all child processes to a single node for centralized

management of resources, D) Using thread-level synchronization techniques within each individual node. The correct answer is A. This question takes the concept of process management learned in a single-machine context and asks the student to extend it to a distributed environment. Option C effectively identifies a common misconception (centralizing defeats distribution), and Option D correctly recognizes that thread-level synchronization addresses only intra-node concerns. However, Option B is too obviously incorrect—"increasing priority" doesn't logically connect to resource allocation or interference prevention across nodes, making it easily eliminated

7.6. Limitations Found

The evaluation revealed several systematic limitations in the question generation process. A clear pattern emerged across cognitive levels: as the taxonomic complexity increased from unistructural to extended abstract, question quality deteriorated. Lower-level questions (unistructural and multistructural) generally demonstrated stronger alignment with their intended cognitive levels. However, higher-level questions, particularly at the relational and extended abstract levels, exhibited significant validity concerns. The correct answers themselves sometimes contained technical inaccuracies or oversimplifications.

Distractor quality also declined at higher cognitive levels. While unistructural questions typically featured distractors targeting genuine misconceptions, extended abstract questions often included options that were either trivially eliminable (violating fundamental principles obvious to any competent student) or failed to reflect realistic reasoning errors students might make when extending concepts to new domains. Additionally, distractor redundancy emerged as a recurring issue, with multiple options sometimes conveying essentially the same incorrect concept, reducing the discriminatory power of the assessment items.

8. Conclusion

This paper presented a system for automatically generating educational quizzes based on the SOLO taxonomy. The system combines local AI models, semantic web technology, and a user-friendly interface to help teachers create better assessments with less effort.

The system is not meant to replace teachers, but to be a tool that saves time and helps create more diverse assessments.

Future work could focus on improving the distractor generation, handling more complex document formats, and adding analytics to track student performance over time. The system could also be extended to support other question types beyond multiple choice, like short answer or essay questions.

Overall, this project shows that combining AI with educational theory can produce useful tools that make teaching and learning more effective. By running everything locally, the system is accessible to anyone without requiring expensive subscriptions or giving up control over their data.

9. References

[1] Chan, C.C., Tsui, M.S., Chan, M.Y. and Hong, J.H., 2002. Applying the structure of the observed learning outcomes (SOLO) taxonomy on student's learning outcomes: An empirical study. *Assessment & Evaluation in Higher Education*, *27*(6), pp.511-527.

[2] Smith, B., 2012. Ontology.

[3] Marcondes, F.S., Gala, A., Magalhães, R., Perez de Britto, F., Durães, D. and Novais, P., 2025. Using ollama. In *Natural Language Analytics with Generative Large-Language Models: A Practical Approach with Ollama and Open-Source LLMs* (pp. 23-35). Cham: Springer Nature Switzerland.

[4] Mitkov, R., 2003. Computer-aided generation of multiple-choice tests. In *Proceedings of the HLT-NAACL 03 workshop on Building educational applications using natural language processing* (pp. 17-22).

[5] Liang, C., Yang, X., Dave, N., Wham, D., Pursel, B. and Giles, C.L., 2018, June. Distractor generation for multiple choice questions using learning to rank. In *Proceedings of the thirteenth workshop on innovative use of NLP for building educational applications* (pp. 284-290).

[6] Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E. and Krusche, S., 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences*, *103*, p.102274.

[7] Lister, R., Simon, B., Thompson, E., Whalley, J.L. and Prasad, C., 2006. Not seeing the forest for the trees: novice programmers and the SOLO taxonomy. *ACM SIGCSE Bulletin*, *38*(3), pp.118-122.

[8] Masse, M., 2011. *REST API design rulebook: designing consistent RESTful web service interfaces*. " O'Reilly Media, Inc.".

[9] Aslam, F.A., Mohammed, H.N. and Lokhande, P.S., 2015. Efficient Way Of Web Development Using Python And Flask. *International Journal of Advanced Research in Computer Science*, *6*(2).

[10] Kreibich, J., 2010. *Using SQLite*. " O'Reilly Media, Inc.".

[11] Pérez, J., Arenas, M. and Gutierrez, C., 2009. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)*, *34*(3), pp.1-45.

[12] Gackenheimer, C., 2015. *Introduction to React*. Apress.